```
    int countLessEqual(int[][] matrix, int target){
        int n = matrix.length;
        int count = 0;
        int row = n-1, col = 0;

        while(row >= 0 && col < n){
            if(matrix[row][col] <= target){
                count += (row + 1);
                col++;
            } else{
                row--;
            }
        }

        return count;
    }
}
```

Sliding Window & Two pointers

O(n^2) to generate all possible substring:

```
for (int i = 0; i < n; i++)
 {
     for (int j = i; j < n; j++)
     {
       substrings.add(str.substring(i, j + 1));
     }
 }
```

Note : problems involving Maximum substring stuff -> gives **hint** of Two pointer and Sliding window.*

1. ***Longest substring with no repeating. LC -3***

class Solution {

```
public int lengthOfLongestSubstring(String s) {
       int[] indexInfo = new int[1000];
       Arrays.fill(indexInfo, -1);
       int end = 0, start = 0;

       int ans = 0;
       while(start < s.length()){
           //seeing this char first time
           if(indexInfo[(int)s.charAt(start)] == -1){
               ans = Math.max(ans,(start - end + 1)); //number of characters in that window
               indexInfo[(int)s.charAt(start)] = start;   //mark index where it was present.
               start++;
           } else {
               // already seen this character.
               int lastSeenPosition = indexInfo[(int)s.charAt(start)];
               while(end <= lastSeenPosition){
                   //remove elements from window
                   indexInfo[(int)s.charAt(end)] = -1;
```

```
                end++;
            }
            indexInfo[(int)s.charAt(start)] = start;
            start++;
        }
    }

    return ans;

  }
}
```

2. *Maximum substring length of 1's if maximum k 0's are allowed to flip.* [LC 1004]

Eg: [1,0,1,1,0,1] k = 2 max substring
Hint: acquire 0 and release when it exceeds k.

```
class Solution {
  public int longestOnes(int[] nums, int k) {
      int zeros = 0;
      int maxLen = 0;
      int start = 0, end = 0;
      while(start < nums.length){
          if(nums[start] == 0){
              //acquire.
              zeros++;
              while (zeros > k){
                  //release
                  if(nums[end] == 0) zeros--;
                  end++;
              }
          }

          int currWindow = start - end + 1;
          maxLen = Math.max(currWindow, maxLen);

          start++;
      }

      return maxLen;
  }
}
```

Rough :
If [0,0] k= 0 then
Zeros = 0
Itr 1 : Start = 0, end = 0
Zeros = 1
Zeros > k so zeros – ⇒ zeros=0 && end =1
currWindow = (start- end +1) ⇒ (0-1 + 1) = 0;

Approach 2 : more optimized.

```
class Solution {
  public int longestOnes(int[] nums, int k) {
      int zeros = 0;
```

```
        int maxLen = 0;
        int right = 0, left = 0;
        int currLen = 0;
        while(right < nums.length){
            if(nums[right] == 0)    zeros++;
            if(zeros > k){
                if(nums[left] == 0)
                    zeros--;

                //maintain the window size.
                left++;
            }
            //till zeros not less than equal k will not update the window size.
            if(zeros <= k){
                currLen =right - left + 1;
                maxLen = Math.max(currLen, maxLen);
            }

            right++;
        }

        return maxLen;
    }
}
```

3. *Fruit Into Basket.* [LC - 904]

You only have two baskets, and each basket can only hold a single type of fruit. There is no limit on the amount of fruit each basket can hold.
Starting from any tree of your choice, you must pick exactly one fruit from every tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.

```
class Solution {
    public int totalFruit(int[] fruits) {
        Map<Integer, Integer> myMap = new HashMap<>();
        int left=0, right=0;
        int maxWindow = 0;

        while(right < fruits.length){
            int currValue = fruits[right];

            if(myMap.get(currValue) != null)
                myMap.put(currValue, myMap.get(currValue) + 1);
            else
                myMap.put(currValue, 1);

            //after insertion if map.size() > 2 shrink left till map.size() != 2;
            while(myMap.keySet().size() > 2){
                int leftValue = fruits[left];
                myMap.put(leftValue, myMap.get(leftValue) - 1);
                left++;

                if(myMap.get(leftValue) == 0)
```

```
                    myMap.remove(leftValue);
            }

            maxWindow = Math.max(maxWindow, right - left + 1);
            right++;
        }

        return maxWindow;
    }
}
```

4. *Longest Repeating Character Replacement* [*LC-424*]

Intuition :

Keep a freq arr.

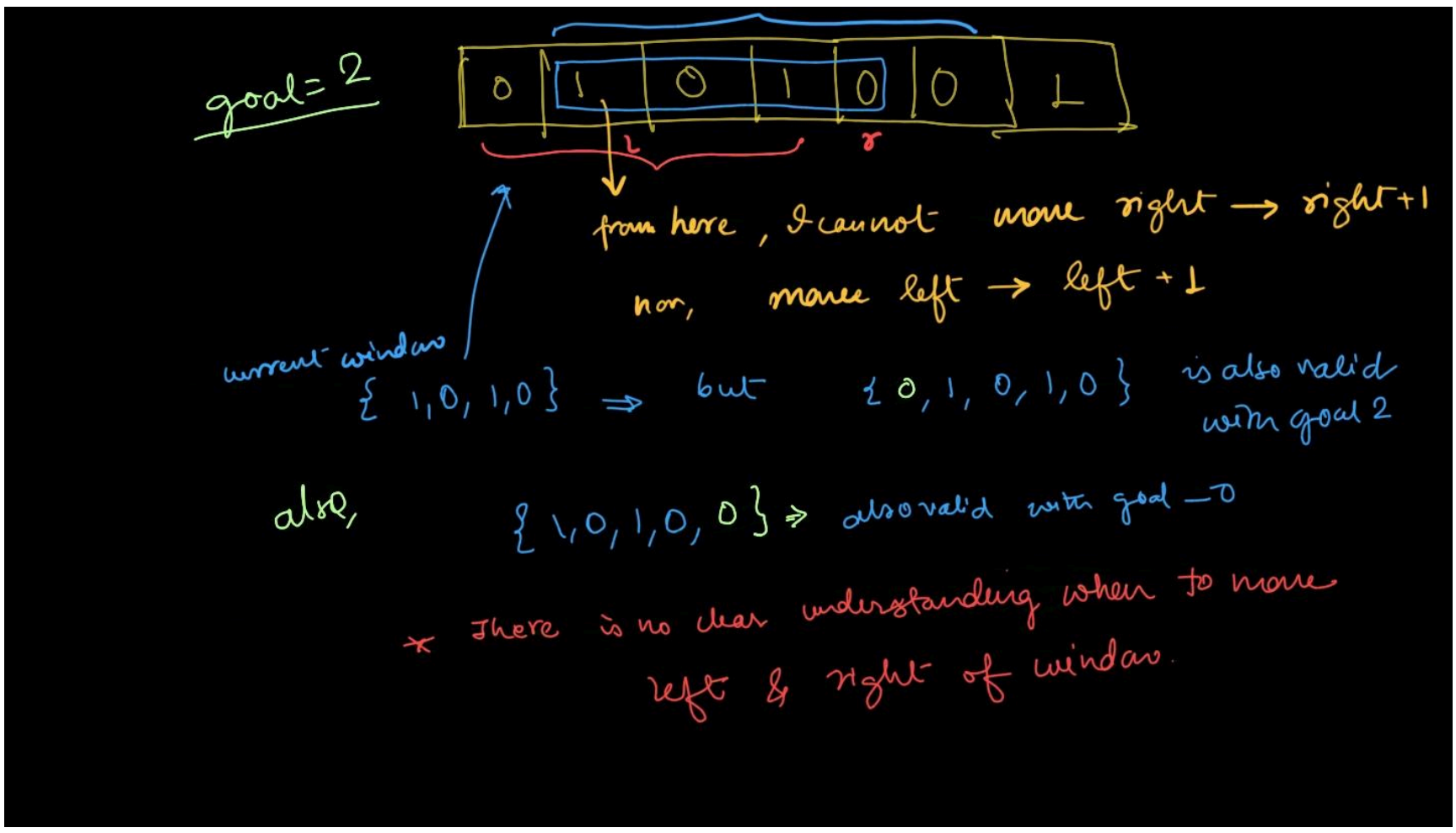
What will be no. of conversions required in a window? → (windowSize - max_freq_of_element)

Eg : window -> "AAABBABC" → conversion required to make all 'A' → windowLength(8) - freq_of_max_occurance(A -> 4) = 4 conversions required. (3B, 1C)

```
class Solution {
    public int characterReplacement(String s, int k) {

        int left = 0, right = 0;
        int[] freq = new int[26];
        int maxWindow = 0;

        while(right < s.length()){
            int currWindow = right - left + 1;
            char curr = s.charAt(right);
            freq[curr - 'A'] += 1;

            //get max freq element
            int maxFreq = getMaxFreq(freq);

            int conversionRequired = currWindow - maxFreq;

            while(conversionRequired > k){
                char leftVal = s.charAt(left);
                freq[leftVal - 'A'] -= 1;
                maxFreq = getMaxFreq(freq);
                left++;

                currWindow = right - left + 1;
                conversionRequired = currWindow - maxFreq;
            }

            maxWindow = Math.max(currWindow, maxWindow);
            right++;
        }

        return maxWindow;

    }
```

```
    int getMaxFreq(int[] freq){
        int ans = 0;
        for(int x : freq)
            ans = Math.max(ans, x);

        return ans;
    }
}
```

5. *BINARY Subarray count with sum equals K -LC-930* **Different and good logic question**



We can easily solve the problem if we require -> ***count of subarrays where <u>sum <= goal</u>***

countSubarr(*sum = K*) => countSubarr(*sum <= K*) - countSubarr(*sum <= K-1*)

```
class Solution {
    public int numSubarraysWithSum(int[] nums, int goal) {
        // res-> subArrCount(sum less or equal goal) - subArrCount(sum less or equal {goal-1})

        int res = countSubarrCountLessOrEqualGoal(nums, goal) -
countSubarrCountLessOrEqualGoal(nums, goal-1);

        return res;
    }

    int countSubarrCountLessOrEqualGoal(int[] nums, int goal){
        if(goal < 0)
            return 0;

        //subarrays with sum (less or equal) goal.
        int left = 0, right= 0, count = 0, windowSum=0;
        while(right < nums.length){
            windowSum += nums[right];
            while(windowSum > goal){
                windowSum -= nums[left];
```

```
                left++;
            }
            count =count + (right - left + 1);
            right++;
        }

        return count;
    }
}
```

6. *Nice Subarrays : Count substrings having count of odd numbers equals K: LC-1248*

Same approach as above.

```
class Solution {
   public int numberOfSubarrays(int[] nums, int k) {
       int res = subarrCountLessThanEqual(nums, k) - subarrCountLessThanEqual(nums, k-1);

       return res;
   }

   int subarrCountLessThanEqual(int[] nums, int target){
       int count = 0;
       int left=0, right=0, windowCount = 0;
       while(right < nums.length){
           if(nums[right] % 2 != 0)    windowCount++;
           while(windowCount > target){
               if(nums[left] % 2 !=0 ) windowCount--;
               left++;
           }
           count = count + (right - left + 1);
           right++;
       }

       return count;
   }
}
```

Practice Questions:

```
/**
 * Q: given array of elements -> what is maximum number of elements we can get such that different between
 *    largest and smallest among them is less than K
 *
 *      arr= [ 18, 4, 6, 1, 2, 7, 15 ]    k=3
 *
 *      what subarrays we can make following the condition
 *      {1,2,4}, {4,6,7}, {15,18}   so max length = 2
 */
import java.util.*;

public class Balanced_Team {
    public static void main(String[] args) {
        int[] nums = {18, 4, 16, 1, 2, 7, 15,17,19,21};
        int K = 7;
        Arrays.sort(nums);
        System.out.println(Arrays.toString(nums));
        //Sliding window

        int L = 0;
```

```
        int ans = 0;
        for(int H = 0; H < nums.length ; H++){
            while(nums[H] - nums[L] > K){
                L++;
            }
            if(H-L+1 > ans){
                System.out.println("H: " + H + ", L: " + L);
            }
            ans = Math.max(ans, H - L + 1);
        }
        System.out.println("Ans : " + ans);
    }
}
```

Prefix Sum problems

1. *Count of subarrays whose sum equals k - LC 560*

We calculate the prefix sum from the start . Standing at an index - i we see - "**have we seen (sum_till_i - k) in map** → if yes

Then till i we can form a subarray with sum k (increment the count)

Else

We mark the sum_till_i visited for future use.eibccbrcfuinkltbjnhjrufcdbhvijglgcfghjllltjj

```
class Solution {
   public int subarraySum(int[] nums, int k) {
       Map<Integer, Integer> map = new HashMap<>();
       map.put(0,1); //sum-0, count-1

       int sum = 0, count = 0;
       for(int num : nums){
           sum += num;
           if(map.containsKey(sum - k)){
               count += map.get(sum - k);
           }
           map.put(sum, map.getOrDefault(sum, 0) + 1); //storing that we have seen the sum.
       }
       return count;
   }
}
```

2. *Maximum Size Subarray Sum Equals LC-325*

Given an integer array `nums` and an integer `k`, return the **length of the longest continuous subarray** whose sum equals `k`.

Instead of count -> we store the first occurrence-index of prefixSum

```
public class Q1_LC_325_MaximumSizeSubarraySumK {
    public static void main(String[] args) {
        int[] nums = new int[]{1,-1,5,-2,3};
        int k = 3;

        //solution
        int n = nums.length;
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, -1); //base case

        int sum = 0 , maxLength =0 ;
```